

CHEMS EDDINE BENTEBOULA

799706

February 2025

REPORT

SENTIMENT ANALYSIS FOR STEAM REVIEWS

SOFTWARE ENGINEERING PROJECT

Abstract

This project focuses on sentiment analysis of movie reviews using the Steam reviews dataset. Multiple machine learning and deep learning models, including Naive Bayes, Logistic Regression, and LSTM, are implemented and evaluated. Feature extraction is performed, and comprehensive data preprocessing is applied. Evaluation metrics include accuracy, precision, recall, F1-score, and confusion matrices. Additionally, visual tools such as WordCloud and sentiment distribution analysis enhance the interpretability of the results. A comparative analysis of model performance highlights the trade-offs between simplicity, computational cost, and predictive accuracy.

1. Introduction

1.1. Motivation

With the explosion of online reviews, extracting valuable insights from textual data is essential. Sentiment analysis allows businesses to assess customer opinions, optimize decision-making, and enhance user experiences. The project focuses on analyzing video games reviews from the steam platform and classify them based on their sentiments as positive or negative.

1.2. Objectives

- Preprocess and analyze text data to prepare it for machine learning models.
- Incorporate advanced visualization tools to improve interpretability.
- Implement traditional and deep learning models for sentiment classification.
- Compare model performance using various evaluation metrics and visualization techniques.

1.3.Dataset Description

Steam is a video game digital distribution service with a vast community of gamers globally. A lot of gamers write reviews on the game page and have the option of choosing whether they would recommend this game to others or not. However, determining this sentiment automatically from the text can help Steam to automatically tag such reviews extracted from other forums across the internet and can help them better judge the popularity of games.

Given the review text with user recommendation and other information related to each game for 64 game titles, the task is to create a test set by making a split from the training set and try to predict whether the reviewer recommended the game titles available in the test set on the basis of review text and other information.

2.Methodology

2.1.Data Preprocessing

- Lowercasing: All text was converted to lowercase to reduce variability.
- Special Characters Removal: Regular expressions were used to remove unwanted characters such as punctuation, numbers, and special symbols.
- Lemmatization: Words were reduced to their base form (e.g., "running" to "run") using WordNetLemmatizer.
- Text Cleaning Function: A reusable Python function was written to automate these preprocessing steps for both training and test sets.

2.2.Data Visualization

Calculating words frequency in our data and visualize it was not an efficient in our case because dealing with comments and reviews specially in the entertainment business through the internet is a complicated task duo to sarcasm and the human creativity of describing such topics.

The method used in this project is a combination of calculating frequent words and and the sentiment score of each word using a library called VADER. It uses a dictionary of words with assigned sentiment scores and a set of rules to analyze text and determine its emotional tone.

The steps of implementation is as follow :

- Takes a DataFrame of movie reviews as input
- Separates positive and negative reviews
- Retrieve the sentiment scores for each word in the reviews using Vader (score range from -4 to +4)

- VADER applies a set of grammatical and syntactical rules to adjust the sentiment scores based on the context of the words (value from -1 to +1)
- Counts word frequencies in the negative reviews.
- Combine both scores for the final word score that will be used to represent the word cloud



2.3.Feature Extraction

Text features were extracted using TF-IDF (Term Frequency-Inverse Document Frequency):

- Parameters used from range (1 , 2) because the main criteria about text binary classification is that negation takes two words most of the time (ex : Not Good)
- Filtering: Terms with frequencies below a minimum threshold were removed to reduce noise.
- TF-IDF Matrix: Transformed the cleaned text into a numerical matrix suitable for machine learning models.

2.4.Machine Learning Models (Naive bias and Logistic regression)

2.4.1.Classic machine learning models :

Used for its efficiency for text classification tasks, especially when the data follows a multinomial distribution and also works well with TF-IDF vectors .

- Implementation: Used MultinomialNB from Scikit-learn.
- Optimization: Hyper parameter tuning (alpha values (0.01 , 0.1 , 1)) to prevent 0 values from being present in the model when testing on an untrained data .
- Implementing GridSearchCV to try the different values of our alpha and select the one that gives best performance
- Implementing GridSearchCV also because while performing hyper parameter tuning it uses cross validation based on the number of parameters so that's also useful to prevent the model from memorizing the pattern of the data .

2.4.2.Deep learning model (LSTM)

Sentences in the reviews text have a sequential structure, where the meaning of a word

depends on its context (previous and next words). LSTMs are designed to remember information across time steps, making them suitable for understanding sentiment based on the sequence of words. Implementation Details:

- Converts the text into numerical format for input into the LSTM model using Keras's Tokenizer.
- Limits the vocabulary to the top 5000 most frequent words (num_words=5000) for computational efficiency.
- Loads pre-trained GloVe embeddings, which provide semantic meaning for each word as a vector.
- Constructs an embedding_matrix for the top 5000 words in the vocabulary.
- Build a sequential LSTM model with the following architecture:
 - Embedding layer (input_dim=5000, output_dim=128) to allows the model to learn word relationships.
 - LSTM layer with 128 units
 - Dense layer with ReLU activation
 - Output layer with sigmoid activation for binary classification (positive or negative sentiment).
- Splits the training data into 5 folds for cross-validation and and early stopping to prevent overfitting.
- Trains the LSTM model for 5 epochs with a batch size of 64 on each training fold.
- Makes predictions on the validation set and calculates the accuracy for each fold to ensures that the model is not overfitting and provides an estimate of its real-world performance.

2.5.Evaluation Metrics

Each model was evaluated using the evaluate model function which calculate:

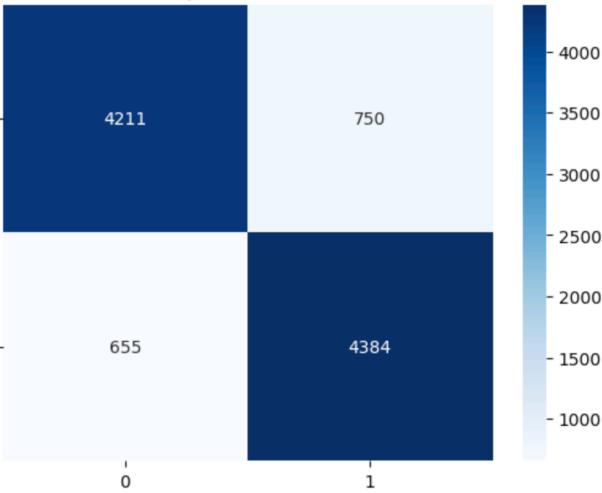
- Accuracy: Percentage of correctly classified reviews.
- Precision, Recall, F1-Score: Used to evaluate class-specific performance, especially for imbalanced datasets.
- Confusion Matrix: Visualized the distribution of true positives, false positives, true negatives, and false negatives.

3.Results and Analysis

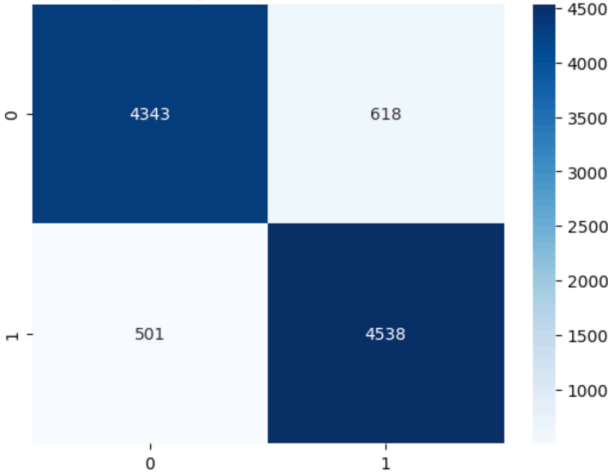
3.1.Quantitative Results

MODELS	ACCURACY	PRECISION	RECALL	F1-SCORE	TRAINING TIME
NAIVE BAYES	82%	83%	83%	82%	15 seconds
LOGISTIC REGRESSION	85%	85%	85%	85%	20 seconds
LSTM	79%	80%	79%	79%	50 minutes

Naive Bayes Confusion Matrix



Logistic Regression Confusion Matrix



LSTM Confusion Matrix



4. Conclusion

This project successfully implemented and evaluated multiple models for sentiment analysis on the steam dataset. Logistic Regression demonstrated the best performance, overall due to the very short amount of time it took for training compared to LSTM and without a significant increase in performance while that also doesn't mean that we wont reach better results if we train it longer because in this case we only trained it for 10 epochs with limited hardware capabilities .

Through this project, we successfully demonstrated the application of both traditional machine learning models and deep learning architectures to sentiment analysis. Each method showcased unique strengths and trade-offs, emphasizing the importance of selecting models based on the specific needs and constraints of the task. This project highlights the importance of preprocessing, feature extraction, and evaluation techniques in achieving reliable results for sentiment analysis tasks.

5. Related Work in Sentiment Analysis

1. “Sentiment Analysis of Movie Reviews” by Pang and Lee (2002)

This seminal work introduced sentiment classification on movie reviews, where the authors experimented with Naive Bayes, maximum entropy models, and SVMs. They highlighted the challenges of text classification for subjective content and demonstrated the effectiveness of machine learning models in sentiment prediction.

2. “Deep Learning for Sentiment Analysis of Movie Reviews” by Maas et al. (2011)

This paper introduced the IMDB dataset used in this project. The authors used word vectors to capture semantic meaning and trained deep learning models for sentiment classification. Their approach demonstrated the value of embeddings for representing textual data.

3. “Learning Word Vectors for Sentiment Analysis” by Mikolov et al. (2013)

The introduction of Word2Vec embeddings revolutionized text representation by enabling words to be mapped into dense vector spaces. These embeddings, often pre-trained, have been widely used in sentiment analysis tasks for improved performance in capturing semantic relationships.

4. “Convolutional Neural Networks for Sentence Classification” by Kim (2014)

This paper introduced a CNN-based architecture for text classification tasks, including sentiment analysis. The model demonstrated the ability to extract high-level features from text, significantly improving performance over traditional methods.